Building the Bible Verse Lookup Tool

Bible Verse Lookup Tool - Build-It-Yourself Lab | Estimated time: 45 min

Objectives

- Understand what a REST API is and how PowerShell talks to one.
- Read and write JSON files as simple local storage.
- Build, piece by piece, a working command-line Bible lookup tool with no prior PowerShell experience.
- End with the exact same tool already installed on this computer - so you can compare your work to the real thing.

Background: what are we actually building?

A **REST API** is just a web address you can request data from, the same way your browser requests a page - except the answer comes back as structured data (**JSON**) instead of a web page. We're going to call the Recovery Version Bible text API at `api.lsm.org`. You send it a verse reference like `John 3:16`, and it sends back the verse text.

Three PowerShell ideas make this possible, and you'll use all three today:

- **Invoke-RestMethod** - the cmdlet that fetches a URL and, if the answer is JSON, automatically turns it into a PowerShell object you can dot into (e.g. `$result.verses`).
- **ConvertTo-Json / ConvertFrom-Json** - turning PowerShell objects into JSON text (to save to a file) and back (to read it again). This is how we'll build "local storage" with nothing but a text file.
- **Functions with parameters** - so typing `verse John 3:16` at the prompt runs your code with `"John 3:16"` as the input.

What the API expects

Endpoint	`https://api.lsm.org/recver/txo.php`
String	The verse reference, e.g. `'John 3:16'`
Out	`json` (otherwise it replies in XML)
appid / token	Your personal credentials from api.lsm.org - also accepted as HTTP Basic Auth
Reply shape	`{ verses: [ {ref, text, urlpfx} ... ], message, copyright }`

Get your own token: Before you start, register at `https://api.lsm.org` to get an `appid` and `token`. Keep them private - treat them like a password. Never paste them into chat, a script you share, or a public file.

Step 1 - Store your credentials safely

We never hard-code a token inside a script (if you ever shared the script, you'd hand over your credentials too). Instead we put them in a small JSON file in your home folder, and have the script read it at runtime.

Create `%USERPROFILE%\.lsm-verse.json` containing:

```
{
  "appid": "YOUR_APPID",
  "token": "YOUR_TOKEN"
}
```

Now write a function that loads it and checks it's actually been filled in:

```
function Get-LsmCredential {
    $configPath = Join-Path $HOME ".lsm-verse.json"
    if (-not (Test-Path $configPath)) {
        Write-Host "Credentials file not found: $configPath" -ForegroundColor Yellow
        return $null
    }
    $config = Get-Content $configPath -Raw | ConvertFrom-Json
    if ($config.appid -like "YOUR_*" -or $config.token -like "YOUR_*") {
        Write-Host "Fill in your real appid/token first." -ForegroundColor Yellow
        return $null
    }
    return $config
}
```

Why check for "YOUR_*": It's a friendly guard rail: if you forget to edit the placeholder file, you get a clear message instead of a confusing API error.

Step 2 - Call the API

Next, a function that takes a reference string, builds the URL, and calls `Invoke-RestMethod`. Note the `-f` format operator building the URL, and `[uri]::EscapeDataString` making sure spaces and colons in the reference don't break the URL.

```
function Invoke-LsmApi {
    param([string]$Reference)
    $config = Get-LsmCredential
    if (-not $config) { return $null }

    [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
    $url = "https://api.lsm.org/recver/txo.php?String={0}&Out=json&appid={1}&token={2}" -f
        [uri]::EscapeDataString("'$Reference'"),
        [uri]::EscapeDataString($config.appid),
        [uri]::EscapeDataString($config.token)

    try {
        return Invoke-RestMethod -Uri $url -TimeoutSec 15
    } catch {
        Write-Host "Request failed: $($_.Exception.Message)" -ForegroundColor Red
        return $null
    }
}
```

Try it now: Dot-source this file (or paste both functions into a PowerShell window) and run `Invoke-LsmApi "John 3:16"`. You should get back an object - try `(Invoke-LsmApi "John 3:16").verses[0].text`.

Step 3 - The 'verse' command, plus clipboard copy

Now wrap that in a function that takes remaining words as the reference (so you can type `verse John 3:16` with no quotes), prints each verse, and copies the text to your clipboard with `Set-Clipboard` - handy for pasting into a message or document.

```
function verse {
    param([Parameter(ValueFromRemainingArguments=$true)][string[]]$Reference)
    if (-not $Reference) { Write-Host "Usage: verse <reference>"; return }

    $result = Invoke-LsmApi -Reference ($Reference -join " ")
    if (-not $result) { return }

    $clipLines = @()
    foreach ($v in $result.verses) {
        Write-Host $v.ref -ForegroundColor Cyan
        Write-Host $v.text
        $clipLines += "$($v.ref) - $($v.text)"
    }
    $clipLines -join "`r`n`r`n" | Set-Clipboard
    Write-Host "(copied to clipboard)" -ForegroundColor DarkGray
}
```

Checkpoint: Run `verse Eph. 4:4-6`. You should see the verse text on screen, and be able to paste it (Ctrl+V) somewhere else.

Step 4 - Local storage: saving a reference, not the text

A JSON file can act as a tiny database - read the existing array from disk (or start with an empty one), append the new item, write the whole array back. The obvious design is to save the verse's *reference and text* together. Don't - check the API's terms of service first.

Read the ToS before you store anything: api.lsm.org's terms (`txo-docs.htm#tos`) explicitly forbid storing any amount of the Recovery Version text for offline use. They also require the `copyright` attribution to be shown wherever verse text is displayed. Any time you're saving data pulled from someone else's API, check what you're actually allowed to keep - "it works" and "it's allowed" are different questions.

The fix: save only the *reference* (a short string like `"John 3:16"`) - never the verse text. When you want to see it again, re-fetch the text live from the API at that moment.

Why a .txt file and not .json: The shipped tool stores one reference per line as `Reference|timestamp` in a plain text file. JSON looks like the obvious choice, but PowerShell 5.1's `ConvertFrom-Json` hands a whole array back as ONE pipeline item, and `ConvertTo-Json` re-wraps piped collections as `{"value":[...],"Count":N}` - so a naive read-append-write cycle nests the file one layer deeper on every single save until it corrupts. A line-based file has none of that, and a non-programmer can open it in Notepad. Match the format to the risk, not to fashion.

Reading and writing it is then boring in the best way:

```
function Get-LsmStorePath { Join-Path $HOME ".lsm-saved-verses.txt" }

function Get-LsmSavedRefs {
    if (-not (Test-Path (Get-LsmStorePath))) { return @() }
    $out = @()
    foreach ($line in (Get-Content (Get-LsmStorePath) -Encoding utf8)) {
        if (-not $line.Trim()) { continue }
        $parts = $line -split '\|', 2
        $out += [PSCustomObject]@{ ref = $parts[0].Trim(); savedAt = $parts[1] }
    }
    return @($out)
}

function Save-LsmVerse {
    param([string]$Reference)
    $saved = @(Get-LsmSavedRefs)
    if ($saved | Where-Object { $_.ref -eq $Reference }) { Write-Host "Already saved."; return }
    $saved += [PSCustomObject]@{ ref = $Reference; savedAt = (Get-Date -Format "yyyy-MM-dd HH:mm:ss") }
    Set-Content -Path (Get-LsmStorePath) -Encoding utf8 -Value (
        $saved | ForEach-Object { "$($_.ref)|$($_.savedAt)" }
    )
    Write-Host "Saved $Reference" -ForegroundColor Green
}

function savedverses {
    $copyright = $null
    foreach ($v in @(Get-LsmSavedRefs)) {
        $result = Invoke-LsmApi -Reference $v.ref   # fetched fresh, every time - never stored
        Write-Host $v.ref -ForegroundColor Cyan
        foreach ($verseObj in $result.verses) { Write-Host $verseObj.text }
        if ($result.copyright) { $copyright = $result.copyright }
    }
    if ($copyright) { Write-Host $copyright -ForegroundColor DarkGray }
}
```

Why @() around Get-Content ...: If only one verse is saved, PowerShell would otherwise hand you a single object instead of an array of one - and += would misbehave. Wrapping in @( ) guarantees you always get an array. This is a classic PowerShell gotcha - remember it.

This design costs one extra API call per saved verse every time you run `savedverses` - a fair trade for staying inside the terms of service, and it has a side benefit: the copyright attribution is now shown automatically, since every fetch returns it.

Step 5 - The 'bible' command: a paged chapter reader

Reading a whole chapter means showing many verses without flooding the screen. The pattern is a `while` loop: show a page, ask what to do next, act on the answer, repeat until the user quits.

```
function bible {
    param([Parameter(ValueFromRemainingArguments=$true)][string[]]$Args)
    $chapterRef = $Args -join " "
    $result = Invoke-LsmApi -Reference $chapterRef
    if (-not $result -or -not $result.verses) { Write-Host "No verses found."; return }
    $verses = @($result.verses)
    $pageSize = [Math]::Max(3, $Host.UI.RawUI.WindowSize.Height - 8)

    $index = 0
    while ($true) {
        Clear-Host
        $pageEnd = [Math]::Min($index + $pageSize, $verses.Count) - 1
        for ($i = $index; $i -le $pageEnd; $i++) { Write-Host $verses[$i].text }

        Write-Host "[N]ext  [P]revious  [S]ave verse  [Q]uit" -ForegroundColor Green
        $choice = (Read-Host ">").Trim()
        switch ($choice.ToUpper()) {
            "N" { if (($pageEnd + 1) -lt $verses.Count) { $index += $pageSize } }
            "P" { $index = [Math]::Max(0, $index - $pageSize) }
            "S" {
                $n = Read-Host "Verse number to save"
                $match = $verses | Where-Object { $_.ref -match ":$n$" } | Select-Object -First 1
                if ($match) { Save-LsmVerse -VerseObj $match }
            }
            "Q" { return }
        }
    }
}
```

Why $Host.UI.RawUI.WindowSize.Height: This reads how tall your current terminal window is, so the page size adapts automatically. Split your terminal pane vertically (tall and narrow) before running `bible` - fewer verses per page, easier to read top to bottom.

A real bug: prefix-matching commands against book names

The first version of this lab used `switch -Regex` with patterns like `'^[Pp]'` - "starts with P". That looks fine until someone types `Psalm 23` to jump to a different chapter: it starts with P, so it silently triggered **Previous page** instead. Same problem for `Numbers` and `Song of Songs` against N and S.

The fix: Match the **whole trimmed input** against the exact letter (`switch ($choice.Trim().ToUpper()) { "N" {...} "P" {...} }`) instead of a prefix. A single letter can only ever mean the command; anything longer falls through to `default`.

The same clash returns with instant keys: The shipped tool later made N/P/S/Q fire on a single keypress (no Enter). That re-opens the identical conflict at a different layer: press **P** meaning "Psalm 23" and you page backwards instead. There is no clever parse that fixes this - one keystroke genuinely cannot be both a command and the start of a word. So the tool adds an explicit escape hatch: `/` opens typing mode, and `/Psalm 23` jumps. Letters that are not commands (`John 4`) still type with no prefix. When two features want the same input, pick a winner and give the loser a deliberate, documented escape - do not try to guess intent.

That `default` case is also where we add the feature you'll want most: if a chapter is short enough to fit on one page, there's no Next/Previous shown - but you can still type. So treat anything that isn't exactly N/P/S/Q/blank as a **new chapter reference** and jump straight to it, without leaving the reader:

```
default {
    $newResult = Invoke-LsmApi -Reference $choice
    if ($newResult -and $newResult.verses -and $newResult.verses.Count -gt 0) {
        $result     = $newResult
        $verses     = @($newResult.verses)
        $chapterRef = $choice
        $index      = 0
    } else {
        Write-Host "Could not find '$choice' - try a format like 'John 4'." -ForegroundColor Red
    }
}
```

Try it: Open a short chapter (e.g. `bible Philemon 1` - one chapter, no Next/Previous appears). At the prompt type `John 4` and press Enter - the reader should jump straight there without you quitting first.

Step 5b - Arrow-key scrolling (why Read-Host isn't enough)

`Read-Host` reads a whole line - it only returns once you press Enter, and it has no idea an arrow key was pressed (the console's line editor swallows arrow keys itself, for moving the cursor left/right within the line). To react to Up/Down *the instant they're pressed*, you need a lower-level read: `$Host.UI.RawUI.ReadKey()`, which returns one key at a time, arrows included.

The tricky part: people still need to type a chapter reference like `John 4` and press Enter. So we build a small function that reads one key at a time in a loop - if it's an arrow and nothing has been typed yet, act on it immediately; otherwise treat it as a character being typed into a buffer, and only hand back the finished text once Enter is pressed:

```
function Read-BibleInput {
    $buffer = ""
    while ($true) {
        $keyInfo = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")
        switch ($keyInfo.VirtualKeyCode) {
            38 { if (-not $buffer) { return @{ Action = "ScrollUp" } } }   # Up arrow
            40 { if (-not $buffer) { return @{ Action = "ScrollDown" } } } # Down arrow
            13 { return @{ Action = "Submit"; Text = $buffer } }            # Enter
            8  {
                if ($buffer.Length -gt 0) {
                    $buffer = $buffer.Substring(0, $buffer.Length - 1)
                    Write-Host "`b `b" -NoNewline   # erase the last character on screen
                }
            }
            default {
                $ch = $keyInfo.Character
                if ($ch -and [int]$ch -ge 32 -and [int]$ch -ne 127) {
                    $buffer += $ch
                    Write-Host $ch -NoNewline   # NoEcho means WE draw each typed character
                }
            }
        }
    }
}
```

38, 40, 13, 8 - what are those?: Virtual-key codes: 38 = Up arrow, 40 = Down arrow, 13 = Enter, 8 = Backspace. These are fixed Windows constants, the same in every language/keyboard layout.

In the reader's loop, use this instead of `Read-Host`, and let `ScrollDown`/`ScrollUp` move the window by exactly **one verse** (instead of a whole `$pageSize`), which is what makes it feel smooth even in a tiny window:

```
$input = Read-BibleInput
switch ($input.Action) {
    "ScrollDown" { if ($index -lt ($verses.Count - 1)) { $index++ } }
    "ScrollUp"   { if ($index -gt 0) { $index-- } }
    "Submit"     { # the N/P/S/Q/jump switch from Step 5, unchanged
    }
}
```

Fallback for safety: Wrap the whole function body in `try { ... } catch { return @{ Action = "Submit"; Text = (Read-Host) } }`. Raw key reads need a real interactive console; this keeps the tool working (just without instant arrows) if it's ever run somewhere that doesn't support it.

Step 5c - Readability: hanging indent and a width/height-aware page

Printing `"$num $text"` works, but two things hurt readability: a long verse wraps at column 0 (the continuation line drifts back under the *number*, not the text), and verses run together with no visual break between them. The fix is a small word-wrap helper with a **hanging indent** - every line after the first is padded with spaces equal to the prefix's width, so it lines up under the verse text:

```
function Get-BibleWrappedLines {
    param([string]$Text, [int]$PrefixLength, [int]$Width)
    $maxLineWidth = [Math]::Max(10, $Width - $PrefixLength - 1)
    $words = $Text -split '\s+'
    $lines = @(); $current = ""
    foreach ($w in $words) {
        if ($current.Length -eq 0) { $current = $w }
        elseif (($current.Length + 1 + $w.Length) -le $maxLineWidth) { $current += " $w" }
        else { $lines += $current; $current = $w }
    }
    if ($current) { $lines += $current }
    return $lines
}
```

Then print the first wrapped line after the coloured verse number, and every line after that indented by the same number of spaces as the prefix, followed by a blank line to separate this verse from the next:

```
$prefix = "{0,3}  " -f $Number
$indent = " " * $prefix.Length
$lines = @(Get-BibleWrappedLines -Text $Text -PrefixLength $prefix.Length -Width $Width)
for ($i = 0; $i -lt $lines.Count; $i++) {
    if ($i -eq 0) { Write-Host $prefix -ForegroundColor Yellow -NoNewline }
    else { Write-Host $indent -NoNewline }
    Write-Host $lines[$i]
}
Write-Host ""   # blank line between verses
```

The @( ) gotcha strikes again: The very first version of this shipped without the `@( )` around `Get-BibleWrappedLines`. Short verses that fit on one line came back as a plain string, not a one-item array - PowerShell silently unwraps single-item collections. Then `$lines[$i]` indexed into that *string* instead of an array, and `$lines[0]` on a string returns its first *character*, not the whole line. Real symptom: verses 2, 3, 4, 8 (long enough to wrap to 2 lines - a genuine multi-item array) displayed fine; verses 1, 5, 6, 7 (short, one line) each showed only their first letter. Same root cause as the `@( )` lesson back in Step 4 - it bit again at a different array boundary. Any time a function might return one item or many, wrap the call in `@( )` at the call site, not just where it's built.

There's a second problem this uncovers: the old page size was just a *count of verses* (`$Host.UI.RawUI.WindowSize.Height - 8`). Once verses can wrap to 2-3 lines each, a fixed verse-count page can overflow a small window. Instead, walk forward adding up the *wrapped line count* (plus one for the spacer) for each verse, and stop once you'd exceed the space available - so the page always fits, however narrow or short the window is:

```
$availableLines = [Math]::Max(3, $termHeight - 9)
$pageEnd = $index; $linesUsed = 0
for ($i = $index; $i -lt $verses.Count; $i++) {
    $need = (Get-BibleWrappedLines -Text $verses[$i].text -PrefixLength 5 -Width $termWidth).Count + 1
    if (($linesUsed + $need) -gt $availableLines -and $i -gt $index) { break }
    $linesUsed += $need
    $pageEnd = $i
}
```

Going back is trickier now: Pages are no longer a fixed size, so "Previous" can't just subtract `$pageSize` - it doesn't know how big the prior page was. Push the current `$index` onto a small list (a stack) every time N is pressed; when P is pressed, pop the last one back off. Simple, and always exactly right.

Step 6 - Wire it into your profile

So these commands exist in every new PowerShell window, dot-source the script from your PowerShell profile. Find your profile path by typing `$PROFILE`, then add:

```
. "C:\path\to\BibleVerseTool.ps1"
```

The real, finished version of this file is already installed on this computer as part of the shipped tool - open it and compare your version line by line. Seeing a working reference alongside your own code is the fastest way to catch what you missed.

Keep ONE copy of the code: Put the functions in `BibleVerseTool.ps1` and let the profile contain nothing but the dot-source line. If you paste the functions into the profile *and* dot-source the file, both definitions load - and the LAST one wins. This actually happened while building this tool: the profile defined the fixed functions, then dot-sourced an older shipped copy at the very end, so every fix appeared to do nothing. The code read correctly and behaved incorrectly, which is the most confusing bug there is.

How to catch it in seconds: When a change to a function seems to have no effect, add a temporary `Write-Host "[dbg] here"` as its first line and re-run. If your debug line never prints, you are not running the function you are editing - go find the other definition with `Get-Command <name> | Select-Object -ExpandProperty ScriptBlock` or by searching your profile for a stray dot-source line.

Checkpoint - prove it all works

1. `verse John 3:16` - verse prints and clipboard shows "(copied to clipboard)".
2. Paste (Ctrl+V) somewhere to confirm the clipboard actually has the verse text.
3. `bible Ephesians 4` - a page of verses appears, sized to your window.
4. Press `N` then `P` - you move forward and back a page.
5. Press `S`, enter a verse number, then `Q` to quit.
6. `savedverses` - the verse you just saved is listed.
7. `bible Philemon 1` (no Next/Previous shown), then type `John 4` and press Enter - the reader jumps to that chapter directly.
8. Narrow your terminal window and open a chapter with long verses (e.g. `bible Romans 8`) - wrapped lines should line up under the text, with a blank line between verses, and the page should never run past the bottom of the window.
9. Shrink your terminal window, run `bible Psalm 119`, and tap the Down arrow a few times - the page should slide down exactly one verse per tap, with no Enter needed.

Quiz

1. What turns the JSON text the API sends back into a PowerShell object you can use with dot notation?
2. Why do we store the appid/token in a separate JSON file instead of typing them directly into the script?
3. What does wrapping a command in `@( )` guarantee, and why does that matter for a single saved verse?
4. In the `bible` function, what stops the page from going past the last verse when you press N?
5. Which cmdlet copies text to the Windows clipboard?
6. Why does `switch -Regex` with pattern `'^[Pp]'` break when someone types `Psalm 23`, and what's the fix?
7. Why can't `Read-Host` react to an arrow key the instant it's pressed, and what do we use instead?
8. A chapter reader shows long verses correctly but short (single-line) verses as just their first letter. What's almost certainly the cause?
9. Why does `savedverses` store only the reference and call `Invoke-LsmApi` again every time, instead of just saving the text once?

Answers

1. `Invoke-RestMethod` - it calls the URL and parses JSON automatically.
2. So the token never appears in a script you might share, copy, or check into somewhere public - and so changing the token doesn't require editing code.
3. It guarantees the result is always an array, even with zero or one items - without it, PowerShell can silently give you a single object instead of a one-item array, breaking += and `.Count`.
4. The check `if (($pageEnd + 1) -lt $verses.Count)` before increasing `$index` - it only advances if there are more verses left to show.
5. `Set-Clipboard`.
6. `'^[Pp]'` matches anything *starting* with P, so "Psalm 23" is misread as the Previous-page command. The fix is matching the whole trimmed input against the exact letter instead of a prefix, so only a bare "P" counts as the command.
7. `Read-Host` only returns once Enter is pressed and the console's own line editor consumes arrow keys for cursor movement. `$Host.UI.RawUI.ReadKey()` reads one raw key at a time instead, so arrows can be acted on immediately.
8. The single-line-wrap function is returning one string instead of a one-item array (the `@( )` gotcha again) - so indexing it with `[0]` returns the string's first character instead of the whole line.
9. Because api.lsm.org's terms of service prohibit storing any amount of the Recovery Version text for offline use. Saving only the short reference and re-fetching the text live keeps the tool useful without ever writing the copyrighted text to disk.

Stretch goals

- Add a `-Lang spa` option to read in Spanish (the API supports it).
- Add an `unsave` command that removes one verse from `.lsm-saved-verses.json` by reference.
- Add a `randomverse` command that picks a random reference from a small built-in list and calls `verse` with it.
- Make `bible` jump straight to a starting verse number instead of always page 1.